

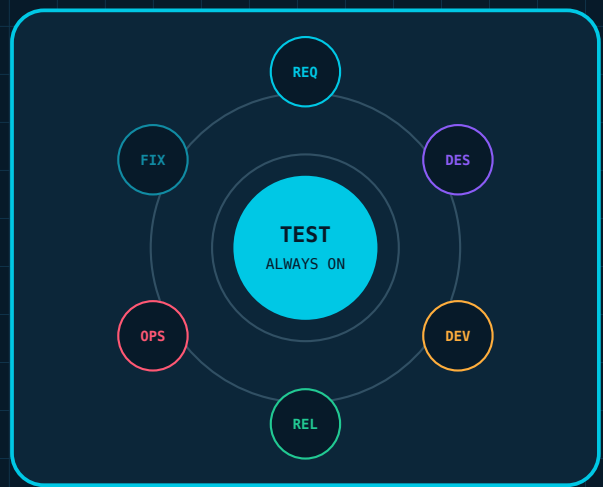
A SZOFTVER ÉLETCIKLUSA ÉS A TESZTELÉS HELYE

Követelménytől az üzemeltetésig: teszt minden szakaszban.

45 PERC

3. TANÓRA

DIÁK MUNKAFÜZET



COURSE_CONTINUE

Premontrei Technikum

Összeállította: Czifra Bálint

v1.0 / 2026

MISSION_03

A mai küldetés: ne csak a végén tesztelj

A PontMentő alkalmazás útján megkeresed, hol előzhető meg, hol fedezhető fel és hol javítható egy eltérés.

FELISMERED

az SDLC fő szakaszait és az egyes szakaszok munkatermékeit.

HOZZÁRENDELED

a követelményhez, tervhez, kódhoz és működő rendszerhez illő teszttevékenységet.

ELKÜLÖNÍTED

a tesztelést, a hibakeresést, a javítást és az újratesttelést.

MEGTERVEZED

egy iskolai alkalmazás tesztpontjait a teljes életciklusban.

MISSION.BRIEF

```
lifecycle = [requirement, design, code, release, operation]
for phase in lifecycle:
    collect(evidence(phase))
    reduce_risk_before_next_phase()
```

SIKERKRITÉRIUM

A 45 perc végén legalább öt életciklus-szakaszhoz helyes teszttevékenységet és egy ellenőrizhető bizonyítékot rendelsz.

SDLC_MAP

Mi az SDLC?

Software Development Life Cycle = szoftverfejlesztési életciklus. A termék útja az igénytől a változtatásig.



TESTING_THREAD.RUN

A tesztelés nem egy külön doboz a fejlesztés végén. Minden szakaszban más munkaterméket vizsgálunk, és bizonyítékot gyűjtünk a következő döntéshez.

SZAKASZ	MUNKATERMÉK	TESZTELŐI KÉRDÉS
Követelmény	leírás, elfogadási feltétel	egyértelmű és ellenőrizhető?
Tervezés	modell, adatfolyam, felület	kezeli a kockázatos eseteket?
Fejlesztés	kód, komponens	a kis egység helyesen működik?
Üzemeltetés	napló, mérés, visszajelzés	mit tanulunk a valós használatból?

STATIC_CHECK

Mit tesztelünk, ha még nincs program?

Követelményt és tervet is vizsgálhatunk futtatás nélkül. A korai kérdés későbbi újramunkát előzhet meg.

REQ-PONT-03

A PontMentő 0 és 100 közötti pontszámot fogad el. Tizedes értéknél a felhasználó területi beállításának megfelelő elválasztót használja, majd elmenti az adatot.

ELLENŐRZŐ KÉRDÉS	IGEN / NEM	PONTOSÍTÁS
A 0 és a 100 is érvényes?	_____	_____
Hány tizedesjegy engedett?	_____	_____
Mi történjen 100,1 esetén?	_____	_____
Mit jelent a 'területi beállítás'?	_____	_____
Milyen hibaüzenet jelenjen meg?	_____	_____

STATIKUS TESZTELÉS

A dokumentumot, modellt vagy kódot futtatás nélkül vizsgáljuk: review, ellenőrzőlista, statikus elemzés.

DINAMIKUS TESZTELÉS

A működő komponenst vagy rendszert futtatjuk, bemenetet adunk és tényleges eredményt figyelünk meg.

KULCSGONDOLAT

A 'még nincs program' nem azt jelenti, hogy még nincs mit tesztelni. A követelmény már tesztalap és munkatermék.

TEST_EVERYWHERE

Tesztelés minden szakaszban

A vizsgálat célja, módszere és bizonyítéka változik, de a kérdés ugyanaz: mit tudunk a kockázatról?

SZAKASZ	MIT VIZSGÁLUNK?	TESZTVÉKENYSÉG	BIZONYÍTÉK
Igény / követelmény	üzleti szabály, elfogadási feltétel	review, példa, kérdés	javított követelmény
Tervezés	felület, adatmodell, interfész	tervreview, prototípuspróba	döntési jegyzet
Fejlesztés	metódus, osztály, komponens	unit teszt, kódreview	teszteredmény
Integráció	komponensek kapcsolata	adatfolyam és interfész teszt	napló, FAIL/PASS
Rendszer / átvétel	teljes felhasználói folyamat	rendszer- és UAT-próba	jegyzőkönyv
Kiadás / üzemeltetés	telepítés, valós működés	smoke, monitorozás, regresszió	mérőszám, hibajegy

MINŐSÉGI KAPU

A teszteléshez mindig legyen tesztalap, megfigyelhető eredmény és rögzített bizonyíték. A 'ránéztem, jónak tűnik' nem elég.

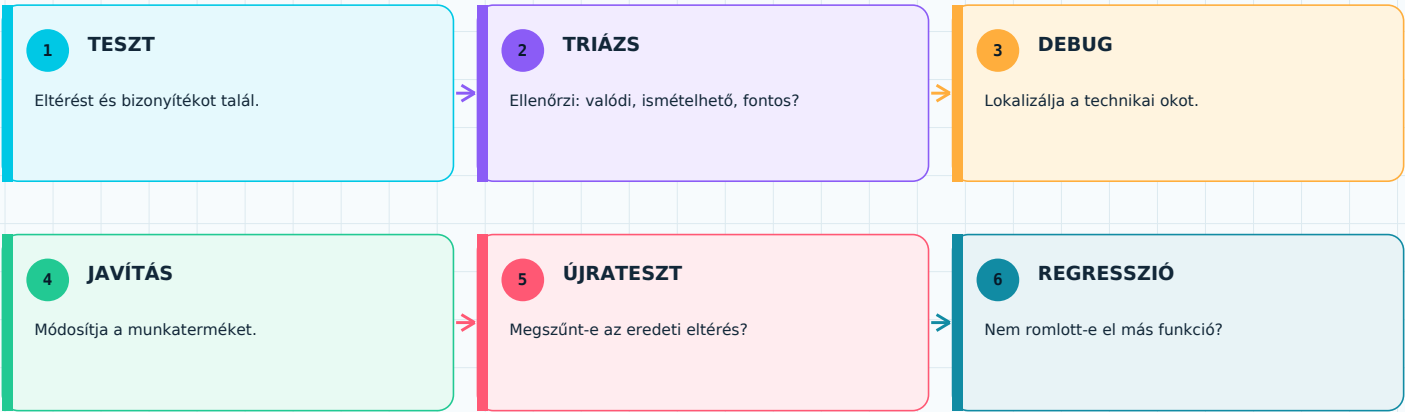
GYORS BESOROLÁS

Írj minden tevékenység mellé szakaszt: K követelmény, T tervezés, F fejlesztés, R rendszer, Ü üzemeltetés.
__ hibás hibaüzenet szövegének review-ja __ parser unit tesztje __ telepítés utáni smoke teszt __ naplófigyelés

FAIL_PIPELINE

Tesztelés, hibakeresés, javítás, újratesztelés

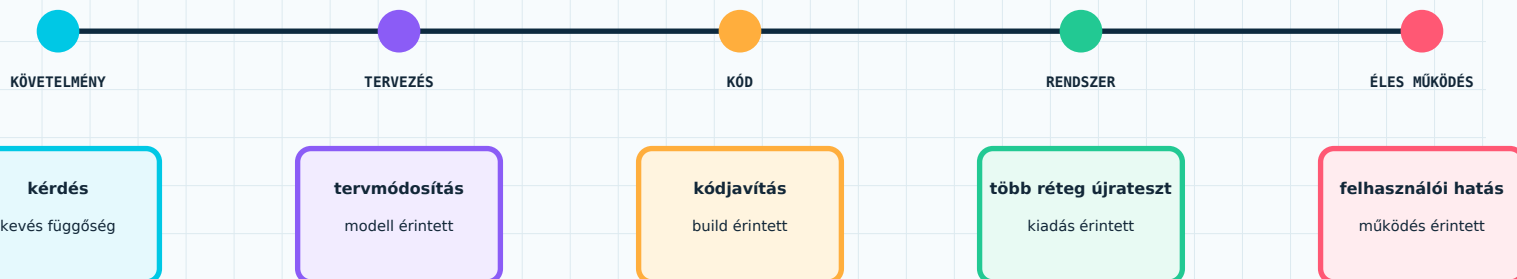
A FAIL nem magyarázza meg automatikusan az okot. A tevékenységek más kérdésre válaszolnak.



EARLY_FEEDBACK

Korai tesztelés és visszacsatolás

Minél korábban kapunk használható visszajelzést, annál kevesebb későbbi elemre terjedhet át ugyanaz a probléma.



SHIFT LEFT

A review-kat, teszttervezést és automatizált ellenőrzéseket korábbi szakaszokba visszük. Nem várunk a teljes rendszerre.

FEEDBACK LOOP

Az üzemeltetés naplói és felhasználói visszajelzései új követelményt, tesztet és fejlesztési döntést indíthatnak.

NEM CSAK BALRA

A 'shift left' nem azt jelenti, hogy az éles működést már nem figyeljük. Korai és késői bizonyítékra egyaránt szükség van.

CASE_FILE

PontMentő v0.3 - bizonyítékcsomag

A rendszer magyar beállításnál nem menti a tizedes pontszámot. Kövesd végig, melyik életciklus-ponton mit tudunk.

E1 // KÖVETELMÉNY

0-100; tizedes érték a felhasználó területi beállítása szerint.

E2 // TERV

A pontszám szöveggént érkezik a felületről; a parser dönt a számmá alakításról.

E3 // TESZT

hu-HU + 75,5 -> ERR-PARSE-17; en-US + 75.5 -> sikeres mentés.

E4 // NAPLÓ

FormatException a pontszám feldolgozásakor; mentési kérés nem indul el.

KÉRDÉS	VÁLASZOD	BIZONYÍTÉK
Melyik szakaszban észlelhető először a kockázat?	_____	E__
Melyik szakaszban jelenik meg megfigyelhető FAIL?	_____	E__
Mi tesztelési tény?	_____	E__
Mi lehet hibakeresési hipotézis?	_____	E__

FORENSIC RULE

Ne ugorj rögtön a javításra. Előbb rögzítsd az elvárást, a környezetet, a lépést és a tényleges eredményt.

CARD_SORT

Életciklus-kártyák: rendezd és kapcsold

Négyfős csapatban rendezzék a kártyákat logikus folyamatba. Minden kártyához válasszatok teszttevékenységet.

KÁRTYA A

A megrendelő kéri a tizedes pontszám mentését.

KÁRTYA B

A csapat pontosítja a hu-HU és en-US elfogadási példákat.

KÁRTYA C

Elkészül a parser terve és a felület adatfolyama.

KÁRTYA D

A fejlesztő megírja a pontszám feldolgozását.

KÁRTYA E

Unit tesztek futnak ponttal, vesszővel és határértékekkel.

KÁRTYA F

A teljes mentési folyamat rendszerpróbája fut.

KÁRTYA G

A verzió kiadás után smoke tesztet kap.

KÁRTYA H

A naplóban megnő az ERR-PARSE-17 esemény.

TEAM_OUTPUT

Sorrend: ____ -> ____ -> ____ -> ____ -> ____ -> ____ -> ____ -> ____

Legkorábbi tesztpont: _____ | Legkésőbbi visszacsatolás: _____

Melyik kártya debugging, és miért? _____

TEST_POINT_MAP

Tesztpont-térkép: hol mit ellenőrzünk?

Töltsétek ki a PontMentőhöz. A 'bizonyíték' oszlopban mindig legyen rögzíthető nyom.

SAKASZ	MUNKATERMÉK	TESZTVÉKENYSÉG	BIZONYÍTÉK
Követelmény	_____	_____	_____
Tervezés	_____	_____	_____
Fejlesztés	_____	_____	_____
Rendszer / kiadás	_____	_____	_____
Üzemeltetés	_____	_____	_____
Változtatás	_____	_____	_____

SZEREPEK

Követelményőr: egyértelműséget vizsgál.
Teszttervező: példát és adatot választ.
Végrehajtó: tényt rögzít.
Riporter: kockázatot és hiányt fogalmaz.

MINŐSÉGGAPU

A csapat csak akkor lép tovább, ha meg tudja nevezni:

1. mit vizsgál;
2. miből tudja az elvárást;
3. milyen bizonyíték marad.

SEGÍTŐ KÉRDÉS

Ha egy szakaszhoz nem tudtok teszttvékenységet írni, tegyétek fel: milyen tévedés juthatna innen a következő szakaszba?

INCIDENT_CANVAS

Hol állítjuk meg a hibát?

Használd a bizonyítéksomagot, és építs korai megelőzést, célzott diagnózist és ellenőrzött javítást.

1. KORAI KÉRDÉS

Mit kérdeznél a követelmény review-ján?

2. TERVELLENŐRZÉS

Milyen döntést keresnél a parser tervében?

3. TESZT

Írj egy vesszős és egy pontos tesztadatot.

4. TÉNY

Fogalmazd meg a megfigyelhető FAIL-t.

5. DEBUG

Írj egy bizonyítandó ok-hipotézist.

6. RETEST

Mit futtatnál a javítás után?

KAPCSOLÓDÁS A 4. ÓRÁHOZ

A 4. órán ugyanezt az esetet tovább bontjuk: gyökérok -> defektus -> kiváltó feltétel -> meghibásodás -> hatás.

EXIT_TICKET

Kilépőkártya - 5 perc

Önállóan válaszolj. A cél annak ellenőrzése, hogy a tesztelést már teljes életciklusban látod-e.

1. kérdés

Mit jelent az SDLC rövidítés magyarul?

2. kérdés

Igaz vagy hamis? Tesztelni csak futtatható programot lehet. Indokold!

3. kérdés

Rendelj egy teszttevékenységet a követelmény, fejlesztés és üzemeltetés szakaszához!

4. kérdés

Mi a különbség a tesztelés és a debugging között?

5. kérdés

Írj egy korai tesztpontot, amely megelőzheti a PontMentő vesszős pontszámhibáját!

Mini szótár
SDLC: szoftverfejlesztési életciklus
debugging: hibakeresés és okfeltárás

Következő állomás
A hiba eredete, aktiválása, látható jelensége és hatása.